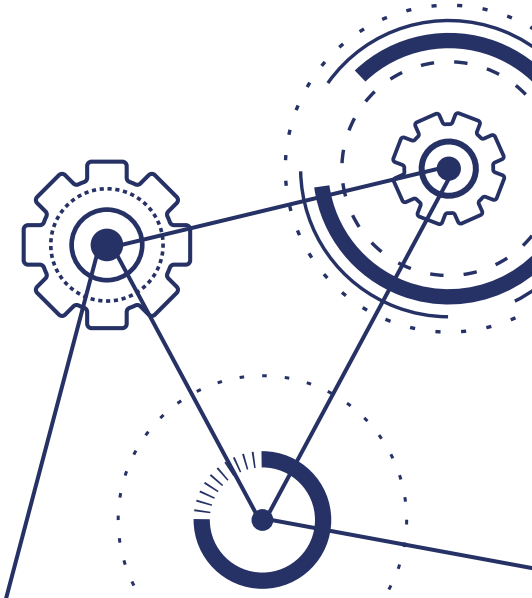
Simulação de deformação elástica

Trabalho de Conclusão de Curso

Carlos Eduardo Valladares da Mota/UFRJ

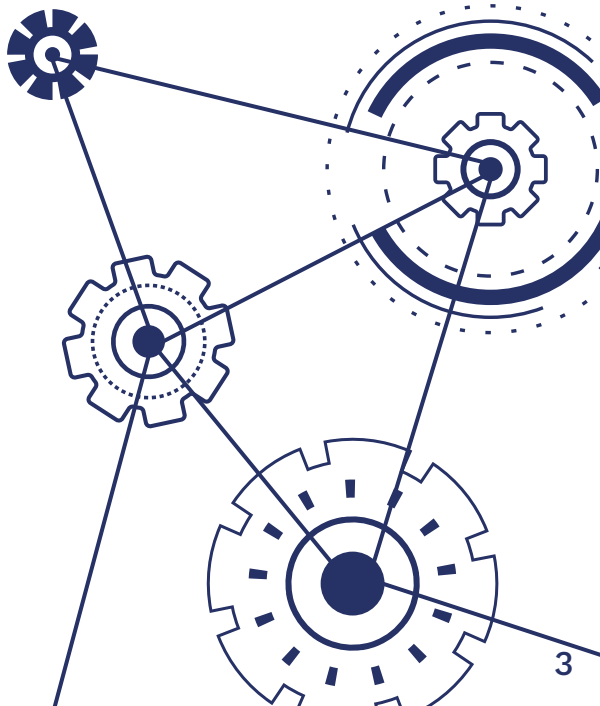


Sumário

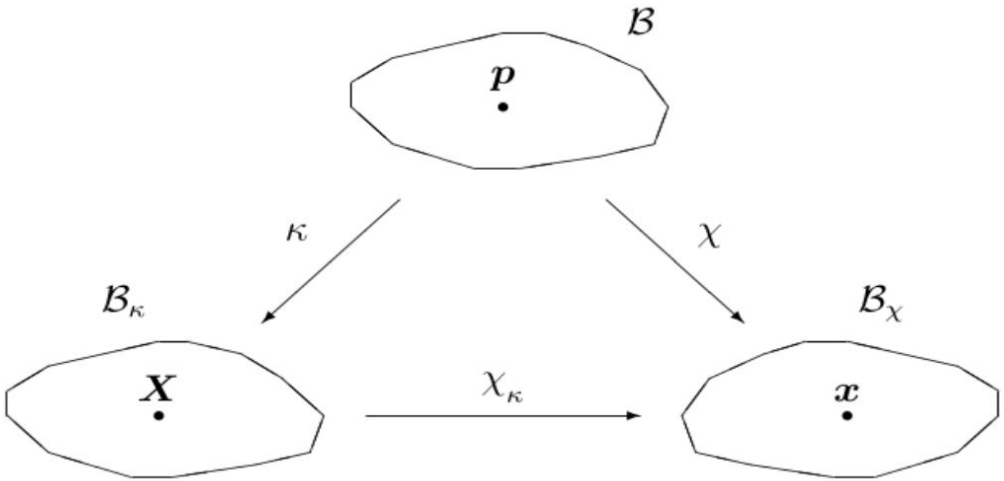
01

O problema

Deformação, material elástico, ALI e cisalhamento



Deformação


$\mathcal{B}$
p
κ
χ
$\mathcal{B}_\kappa$
$\mathcal{B}_\chi$
X
χ_κ
x

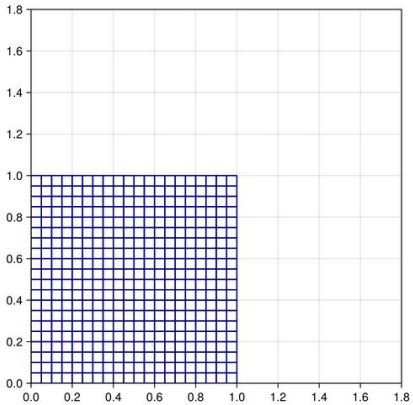
Deformação

$$\boldsymbol{x} = \mathcal{X}(\kappa^{-1}(\boldsymbol{X}))$$

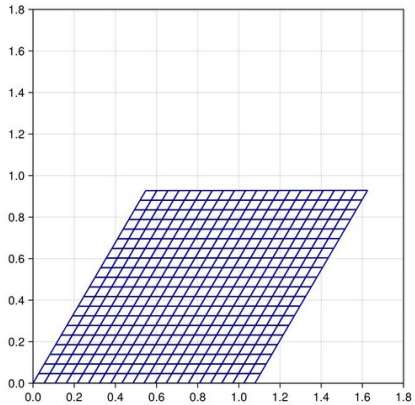
$$F_\kappa = \nabla_{\boldsymbol{X}} \mathcal{X}_\kappa$$

$$[F] = \begin{bmatrix} \frac{\partial x_1}{\partial X_1} & \frac{\partial x_1}{\partial X_2} & \frac{\partial x_1}{\partial X_3} \\ \frac{\partial x_2}{\partial X_1} & \frac{\partial x_2}{\partial X_2} & \frac{\partial x_2}{\partial X_3} \\ \frac{\partial x_3}{\partial X_1} & \frac{\partial x_3}{\partial X_2} & \frac{\partial x_3}{\partial X_3} \end{bmatrix}$$

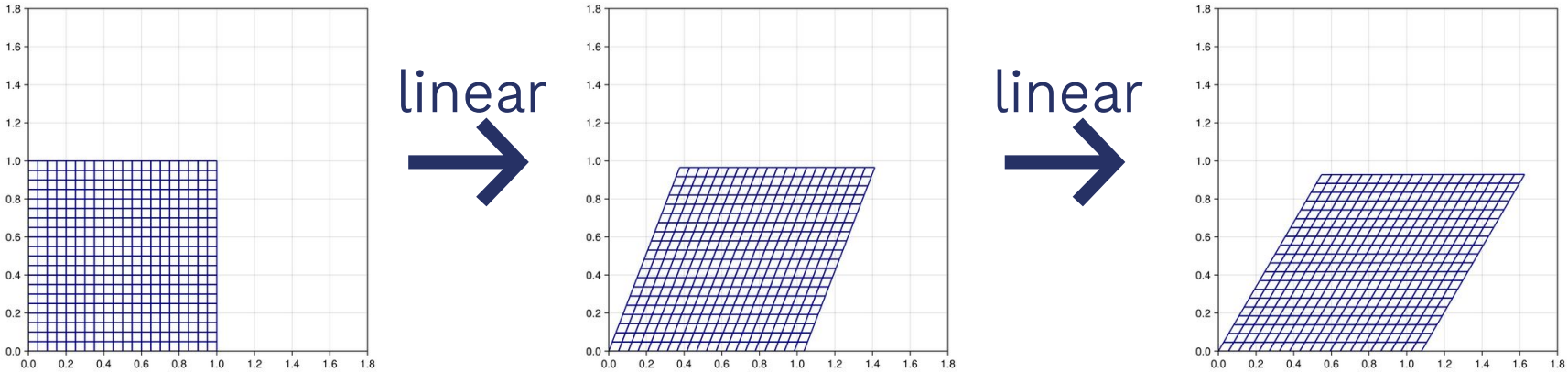
Deformação



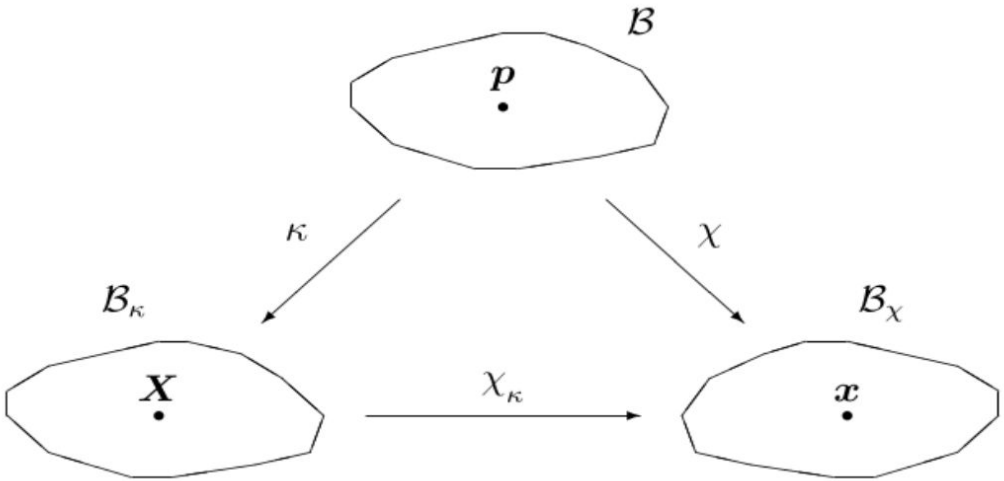
não-linear



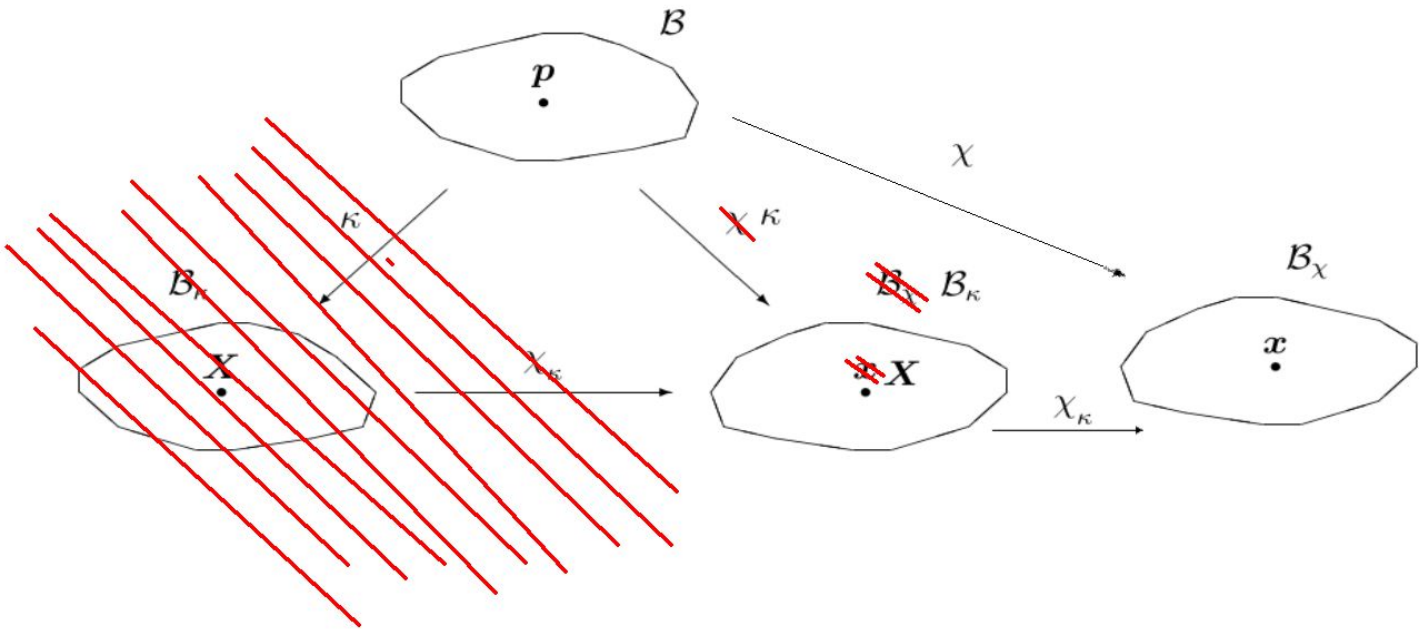
Aproximações Lineares Incrementais (ALI)



Deformação


$\mathcal{B}$
p
κ
χ
$\mathcal{B}_\kappa$
$\mathcal{B}_\chi$
X
χ_κ
x

Deformação


$\mathcal{B}$
p
χ
κ
$\chi \circ \kappa$
$\mathcal{B}_\kappa$
$\mathcal{B}_\chi$
X
χ_κ
x

Pequena Deformação

$$\boldsymbol{u} = \mathcal{X}_\kappa(\boldsymbol{X}) - \boldsymbol{X}$$

$$\nabla_{\boldsymbol{X}} \boldsymbol{u} = \nabla_X \mathcal{X}_\kappa(\boldsymbol{X}) - \nabla_X \boldsymbol{X}$$

$$H = F - I \iff F = I + H$$

$$[H] = \begin{bmatrix} \dfrac{\partial u_1}{\partial X_1} & \dfrac{\partial u_1}{\partial X_2} & \dfrac{\partial u_1}{\partial X_3} \\ \dfrac{\partial u_2}{\partial X_1} & \dfrac{\partial u_2}{\partial X_2} & \dfrac{\partial u_2}{\partial X_3} \\ \dfrac{\partial u_3}{\partial X_1} & \dfrac{\partial u_3}{\partial X_2} & \dfrac{\partial u_3}{\partial X_3} \end{bmatrix}$$

Material elástico

*

vs

Em termos matemáticos

$$\tilde{\mathcal{F}}(F) = s_1 B(\tau) + s_2 B(\tau)^{-1}$$

Material
Mooney-Rivlin

$$T(t) = \mathcal{F}_{\kappa_0}(F) = -pI + \tilde{\mathcal{F}}(F)$$

$$T(\tau) = T(t) + L(F(t))[H]$$

$$L(F(t))[H] = -\beta \mathrm{tr}(H) I + s_1(HB + BH^T) - s_2(H^T B^{-1} + B^{-1} H)$$

ALI - Atualização nos passos

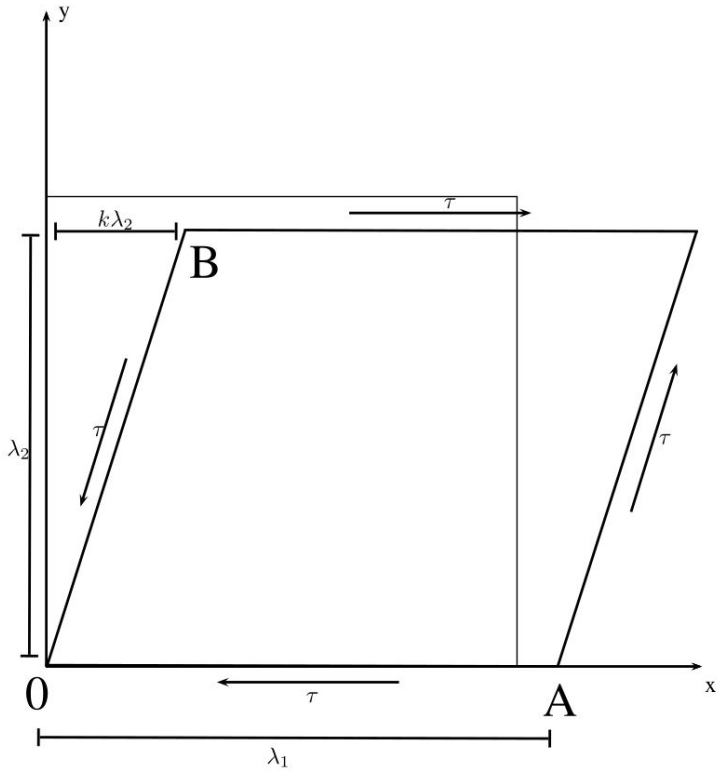
$$x_{n+1} = \chi(X, t_{n+1}) = x_n + \boldsymbol{u}(x_n, t_{n+1})$$

$$F(x_{n+1}, t_{n+1}) = (I + H(x_n, t_{n+1}))F(x_n, t_n)$$

$$T(x_{n+1}, t_{n+1}) = T(x_n, t_n) + L(F(x_n, t_n))[H(x_n, t_{n+1})],$$

Cisalhamento puro

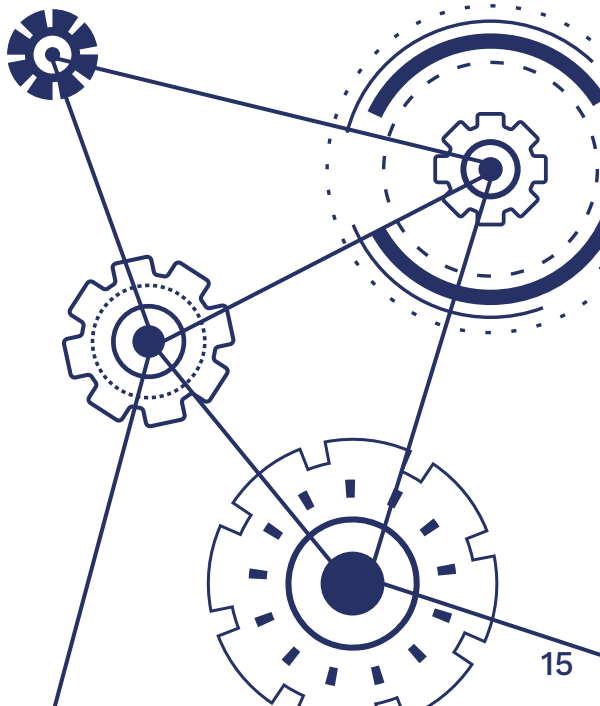
By Kotivalo - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=44782270



02

Formulações

Forte, fraca e matricial.



Formulação Forte

$$\begin{cases} -div(T_\kappa) = 0 \text{ em } \Omega \\ T_\kappa \boldsymbol{n}_\kappa = f \text{ em } \Gamma_1 \\ u(\boldsymbol{x}, \tau) = g \text{ em } \Gamma_2 \end{cases}$$

Formulação Fraca

$$\int_\Omega \frac{\partial u_k}{\partial x_k} T_{ij} \frac{\partial v_i}{\partial x_j}\, d\Omega - \int_\Omega T_{ik} \frac{\partial u_j}{\partial x_k} \frac{\partial v_i}{\partial x_j}\, d\Omega + \int_\Omega L_{ijkl} \frac{\partial u_k}{\partial x_l} \frac{\partial v_i}{\partial x_j}\, d\Omega = \int_{\Gamma_{1i}} f_i v_i\, d\Gamma - \int_\Omega T_{ij} \frac{\partial v_i}{\partial x_j} d\Omega$$

↓

$$a(u, v) = b(v)$$

↓

Formulação Matricial

$$a(w^h, v^h) = b(v^h) - a(g^h, v^h) \ \forall v^h \in \mathcal{V}^h$$

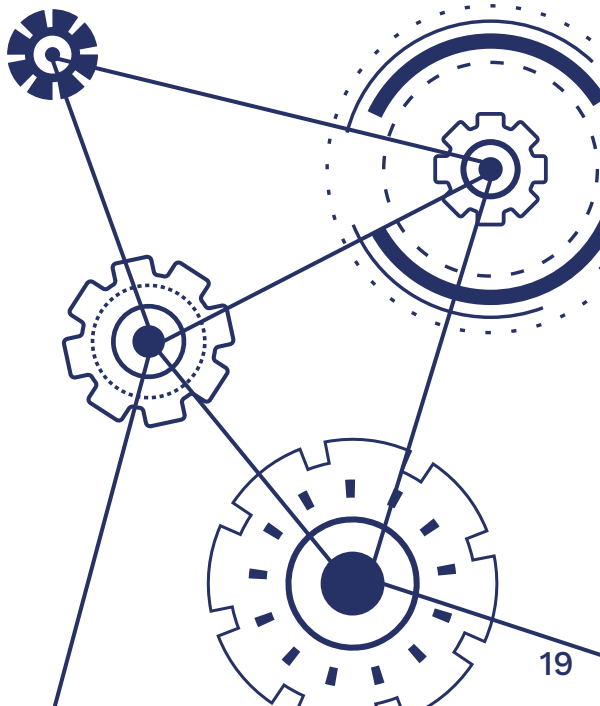
$$K^e_{ab} = \int_{\Omega^B} T_{ri} \frac{\partial \phi^e_a}{\partial \xi_k} \frac{\partial \xi_k}{\partial x_i} \frac{\partial \phi^e_b}{\partial \xi_l} \frac{\partial \xi_l}{\partial x_s} det(J) d\xi_1 d\xi_2 - \int_{\Omega^B} T_{ri} \frac{\partial \phi^e_a}{\partial \xi_l} \frac{\partial \xi_l}{\partial x_s} \frac{\partial \phi^e_b}{\partial \xi_k} \frac{\partial \xi_k}{\partial x_i} det(J) d\xi_1 d\xi_2 + \int_{\Omega^B} L_{risj} \frac{\partial \phi^e_a}{\partial \xi_k} \frac{\partial \xi_k}{\partial x_i} \frac{\partial \phi^e_b}{\partial \xi_l} \frac{\partial \xi_l}{\partial x_j} det(J) d\xi_1 d\xi_2$$

$$F^e_a = \sum_{b=1}^{4} \int_{\Gamma^B_{1j}} \phi^e_a \phi^e_b f_{jb} \ det(J) d\xi_1 d\xi_2 - \int_{\Omega^B} T_{ri} \frac{\partial \phi^e_a}{\partial \xi_k} \frac{\partial \xi_k}{\partial x_i} det(J) d\xi_1 d\xi_2 - \sum_{b=1}^{4} K^e_{ab} \ g_{jb}$$

03

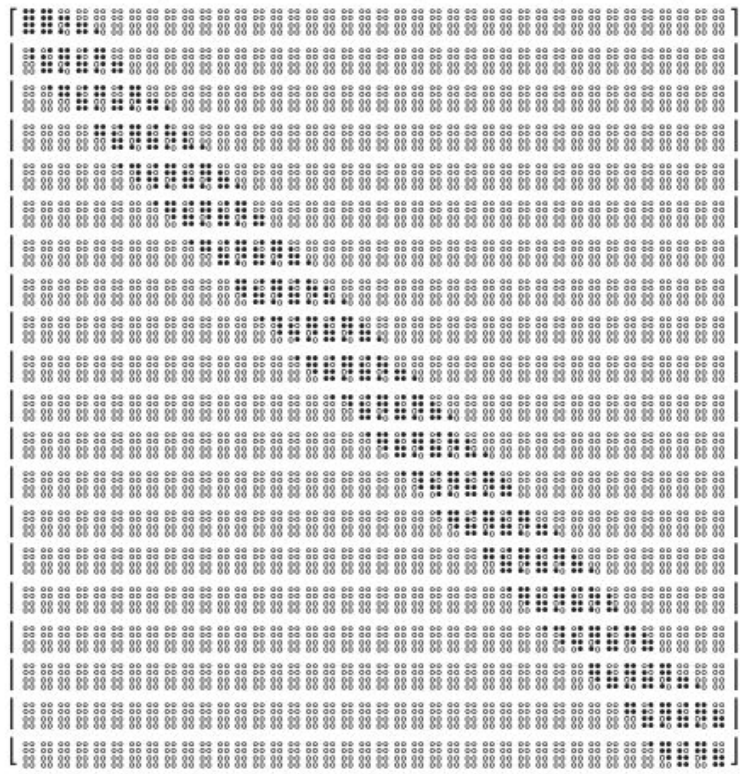
Implementação

Escolhas, processos e testes.



Estruturas

```
860×860 SparseMatrixCSC{Float64, Int64} with 14446 stored entries:
```



Armazenamento da K:

SparseMatrixCSC

Estruturas

```
\
```

```
\ (generic function with 190 methods)
```

```
K, F = monta_K_F_global(params,
c = K\F
```

```
LinearAlgebra.lu
```

```
lu (generic function with 20 methods)
```

Solução do S.L.

\(A, b)

Testes (*debugging*)

```
F_defᵉ = F_def[e]
Tᵉ = T[e]
#Kᵉ = ones(8, 8)
s1, s2, β =  params
h  = -s2/s1
# Atualizações de p?? Tr(H) ≈ 0 sempre, pode ficar de fora
p = 1-h
_params = [s1, s2, p, β]
Kᵉ = monta_K_local(_params, Xᵉ, F_defᵉ, Tᵉ, P, W)

#Fᵉ = ones(8)
presc_map = dirichlet_map(presc, e, LG)
Fᵉ = monta_F_local(f, g, presc_map, e_boundaries, Xᵉ, Kᵉ, F_defᵉ, Tᵉ, P, W)

display(Kᵉ)
display(Fᵉ)
```

Testes (*debugging*)

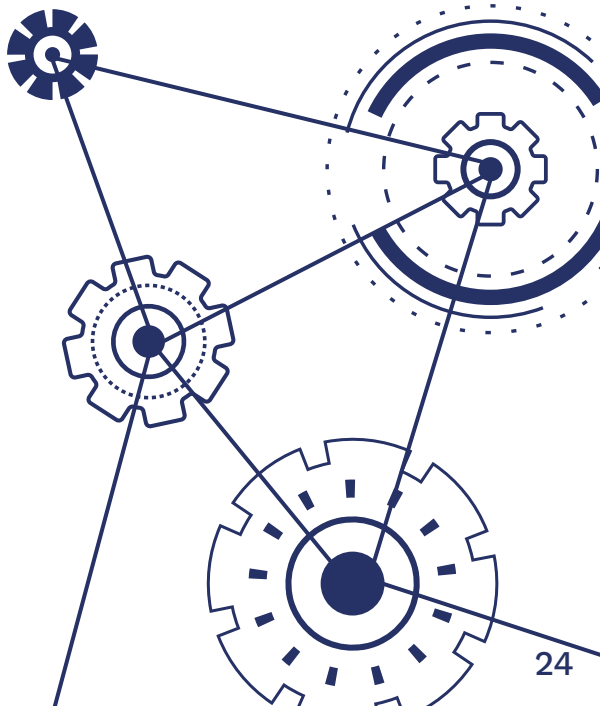
```
p1 = pts_fronteira[1]
p2 = pts_fronteira[2]
p3 = pts_fronteira[3]
p4 = pts_fronteira[4]

P, W = legendre(2)

e_fronteiras = [
    getQuadSideElements(Nx1, Nx2, 1),
    getQuadSideElements(Nx1, Nx2, 2),
    getQuadSideElements(Nx1, Nx2, 3),
    getQuadSideElements(Nx1, Nx2, 4),
]

for l in fronteiras[4]
    print(bound4(u[l, :]), " ")
end
```

```
pts_front = collect(unique(Iterators.
println(pts_front)

for pt in pts_front
    println(f(u[pt, :]))
end

for pt in pts_front
    println(u[pt, :])
end
```

04

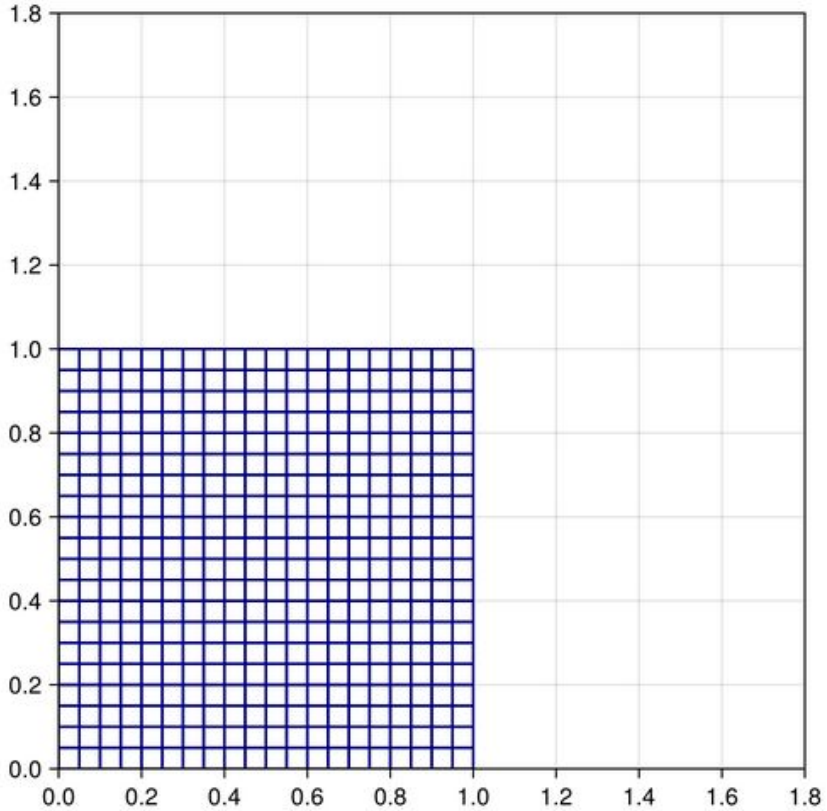
Resultados

Simulação, gráficos e comportamento do erro



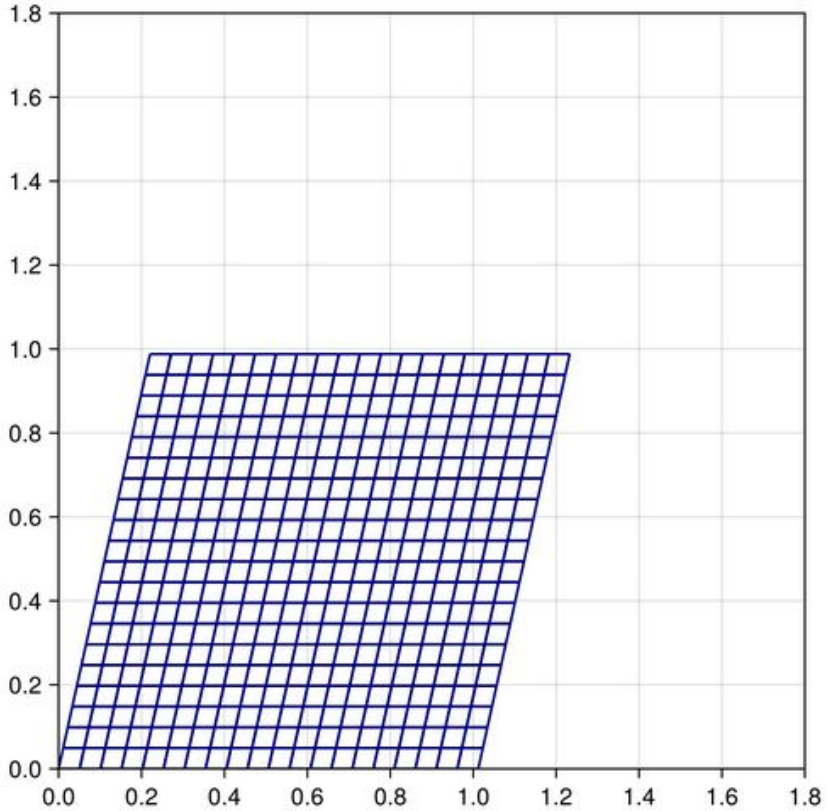
Simulação

Passo 0



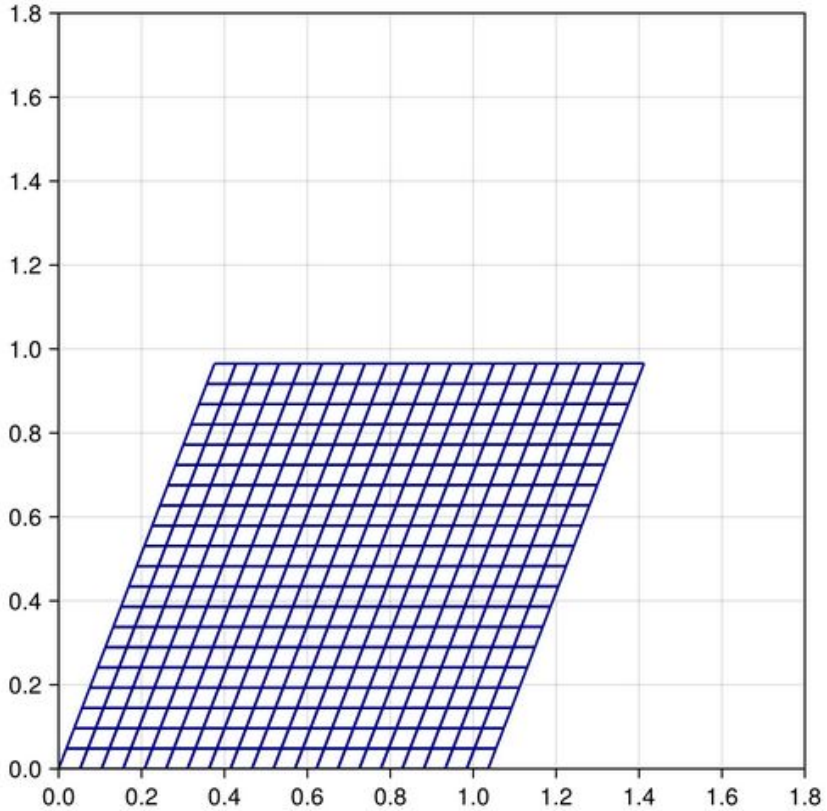
Simulação

Passo 60



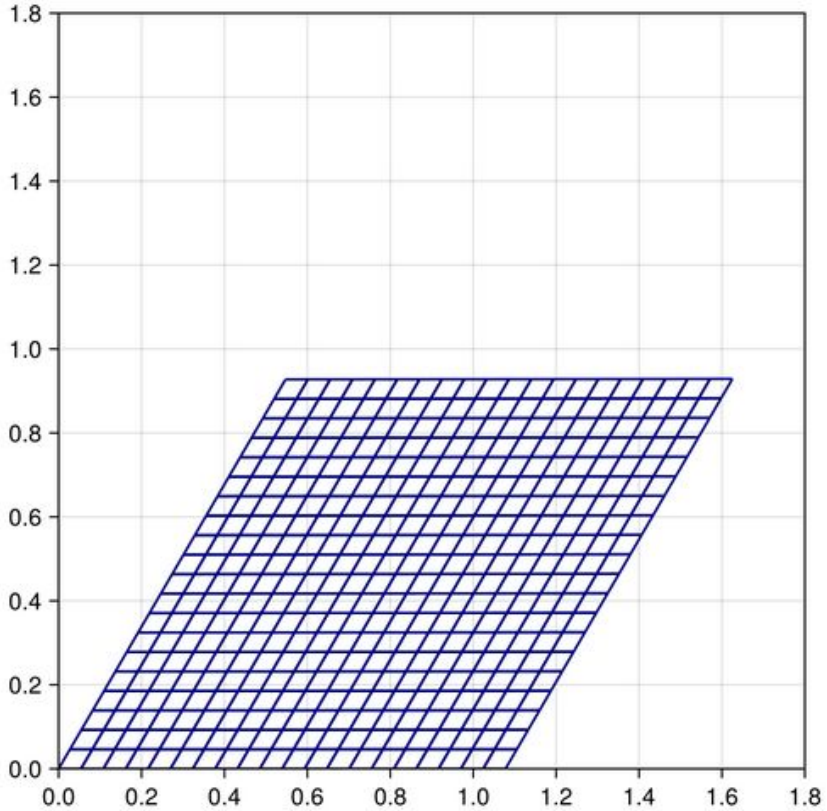
Simulação

Passo 100



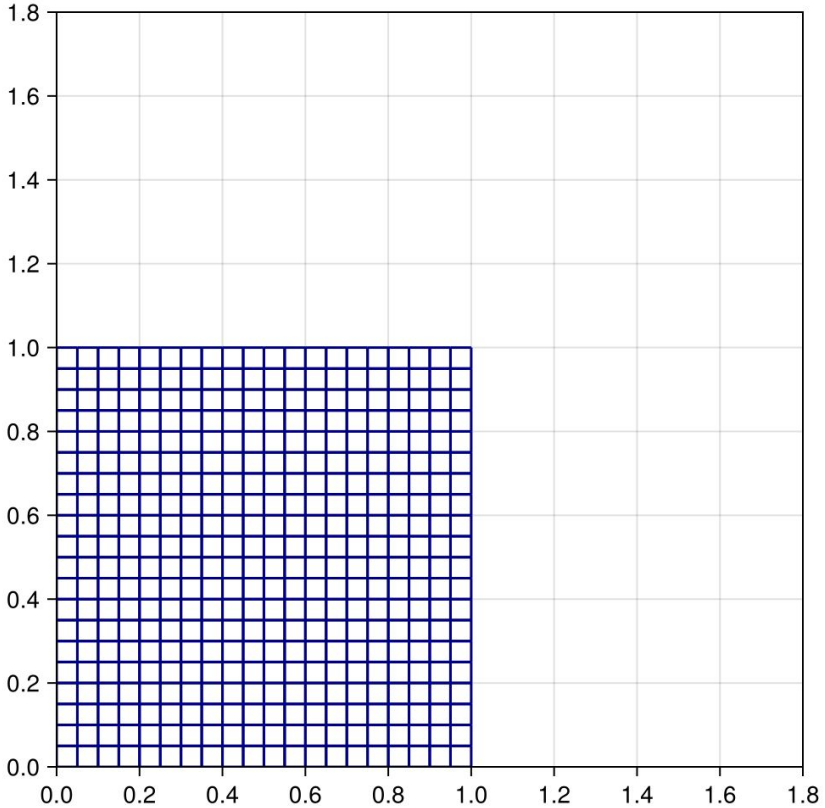
Simulação

Passo 140



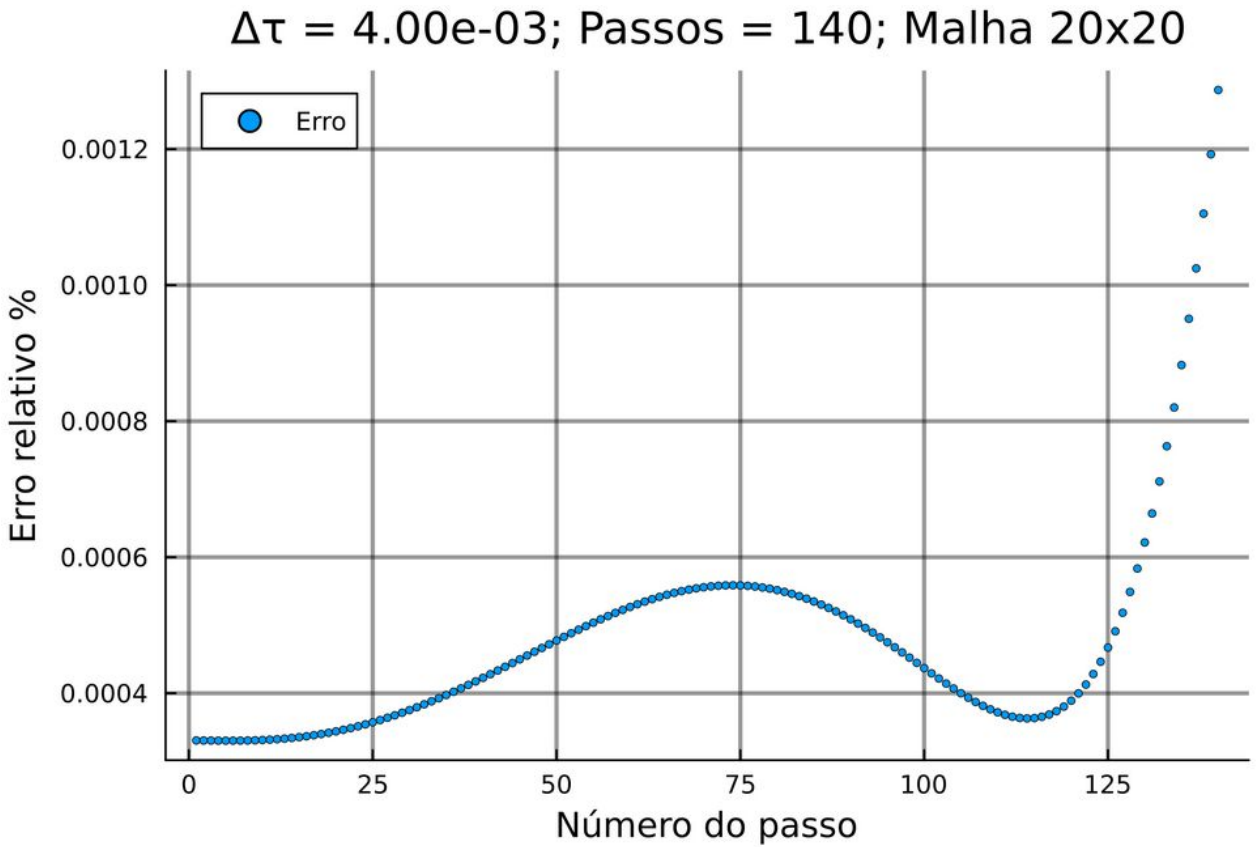
Simulação

Todos os Passos (GIF)

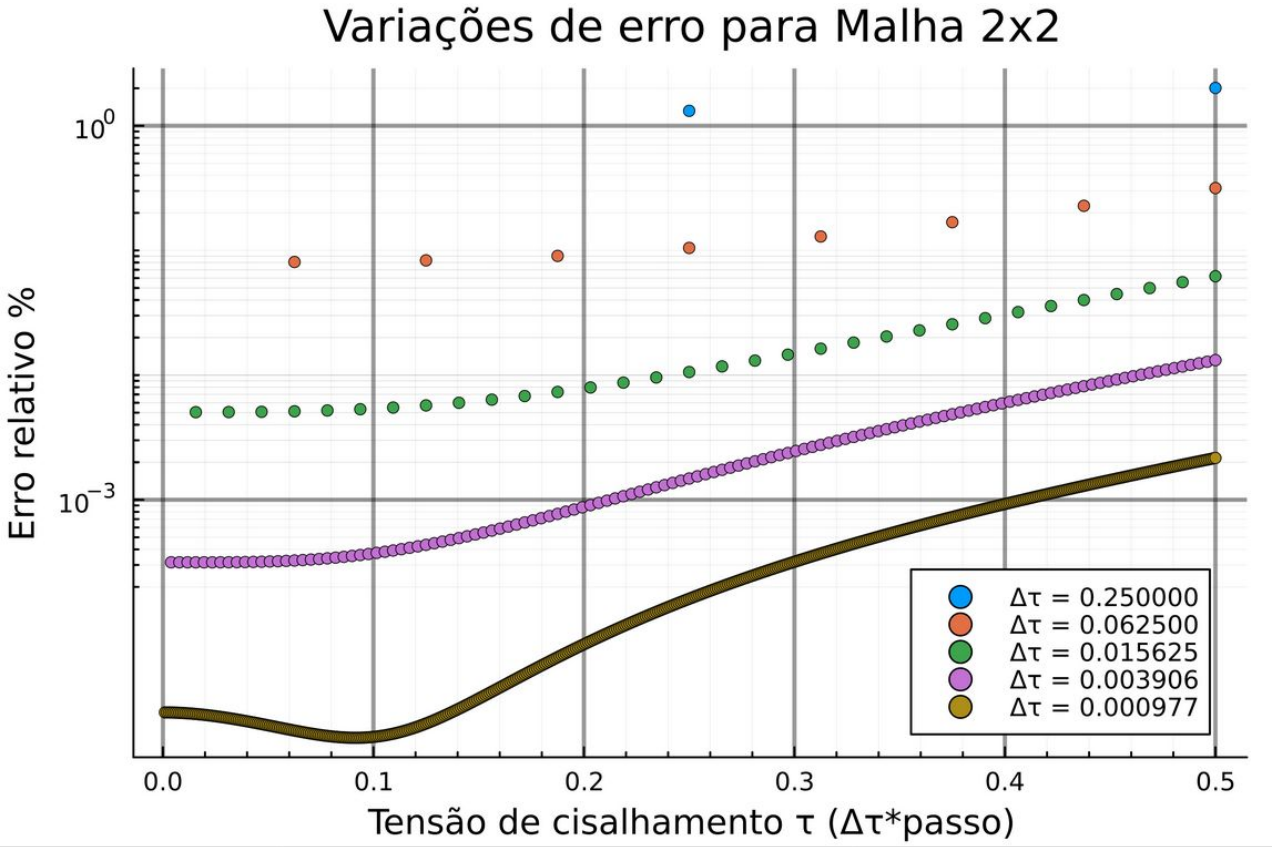


Gráficos

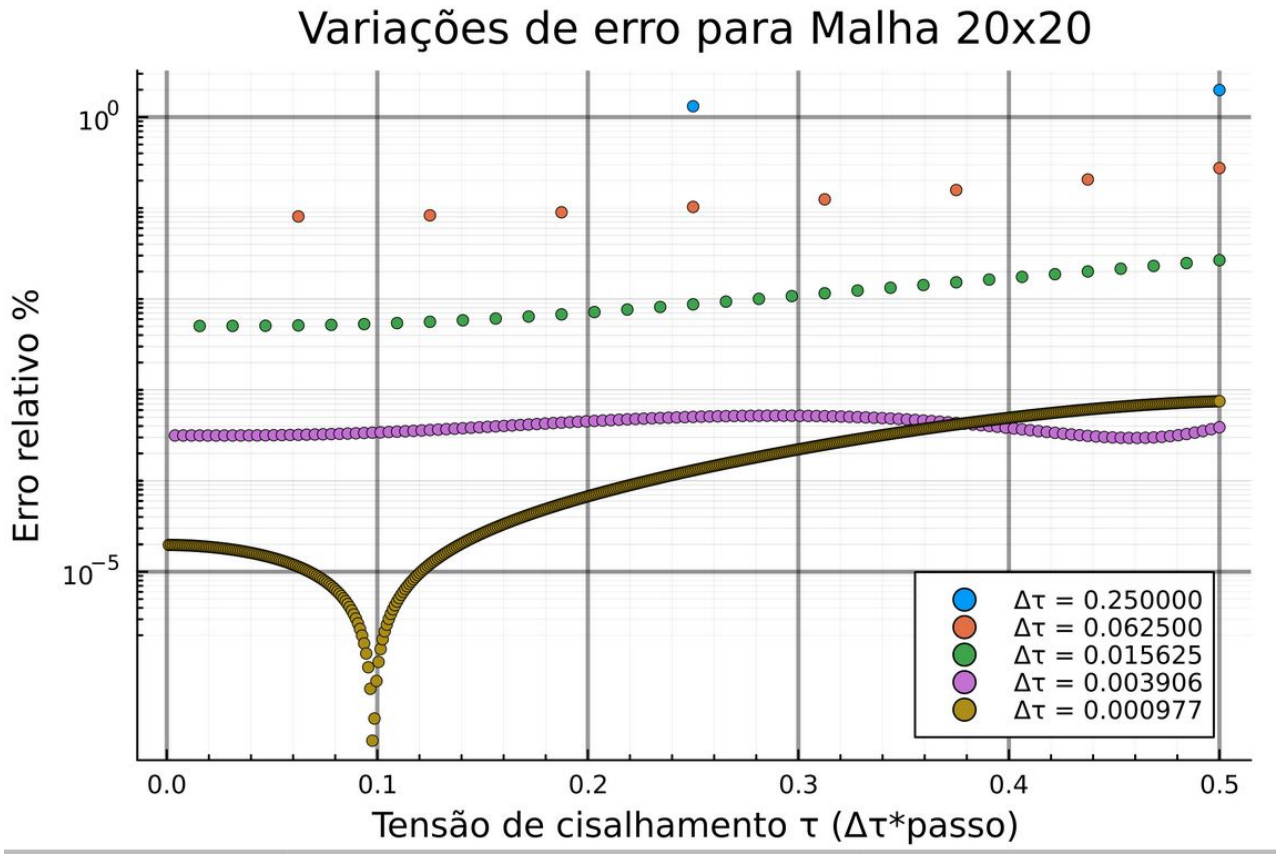
Erro relativo (exp. ref.)



Gráficos


Variações de erro para Malha 2x2
Erro relativo %
10^0
10^{-3}
0.0
0.1
0.2
0.3
0.4
0.5
Tensão de cisalhamento τ (Δτ*passo)
Δτ = 0.250000
Δτ = 0.062500
Δτ = 0.015625
Δτ = 0.003906
Δτ = 0.000977

Gráficos


Variações de erro para Malha 20x20
Erro relativo %
10^0
10^{-5}
0.0
0.1
0.2
0.3
0.4
0.5
Tensão de cisalhamento τ (Δτ*passo)
Δτ = 0.250000
Δτ = 0.062500
Δτ = 0.015625
Δτ = 0.003906
Δτ = 0.000977

Comportamento do Erro

Tabela 1 – Erro relativo máximo dos experimentos

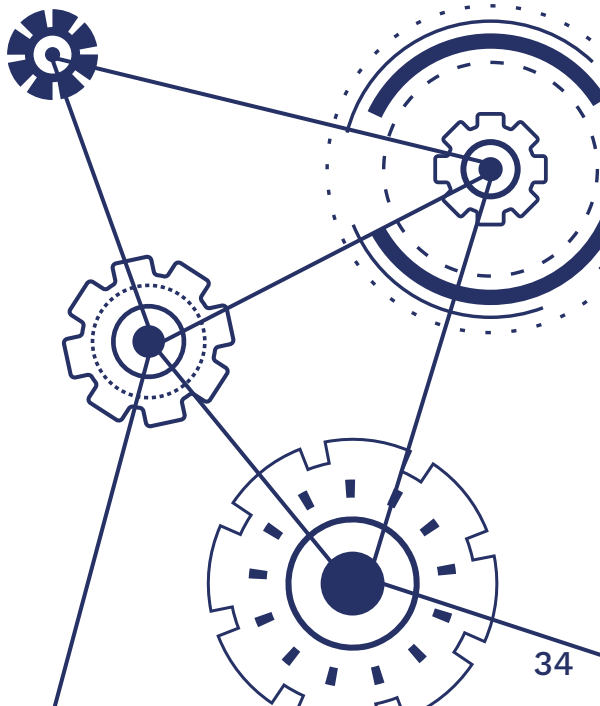
$\Delta\tau$ \Nº Elementos	2×2	5×5	10×10	20×20	40×40
$0.5/2^1$	2.0115%	2.0100%	2.00468%	1.98605%	1.93680%
$0.5/2^3$	0.3164%	0.3133%	0.30336%	0.27530%	0.23003%
$0.5/2^5$	0.0620%	0.0575%	0.04587%	0.02671%	0.01411%
$0.5/2^7$	0.0131%	0.0093%	0.00387%	0.00038%	0.00056%
$0.5/2^9$	0.0021%	0.0003%	0.00051%	0.00075%	0.00079%

Fonte: Autoria própria.

05

Trabalhos Futuros

Otimização, inclusão em biblioteca



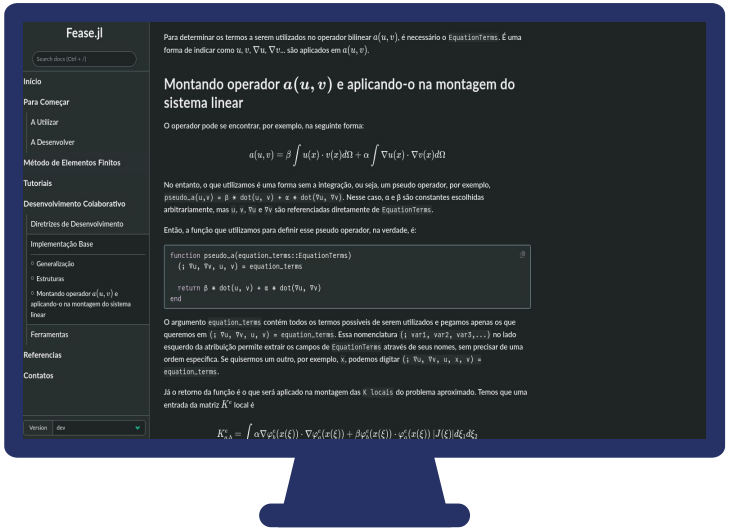
Otimizar código

ΔT \ Nº e	2x2	5x5	10x10	20x20	40x40
$0.5/2^1$	(<1seg)	(<1seg)	(2seg)	(7seg)	30seg
$0.5/2^3$	(<1seg)	(2seg)	(7seg)	(30seg)	2min
$0.5/2^5$	(2seg)	(7seg)	(30seg)	(2min)	7min
$0.5/2^7$	(7seg)	(30seg)	(2min)	7min	30min
$0.5/2^9$	(30seg)	(2min)	(7min)	30min	2H

Biblioteca (e Mestrado)

Fease.jl

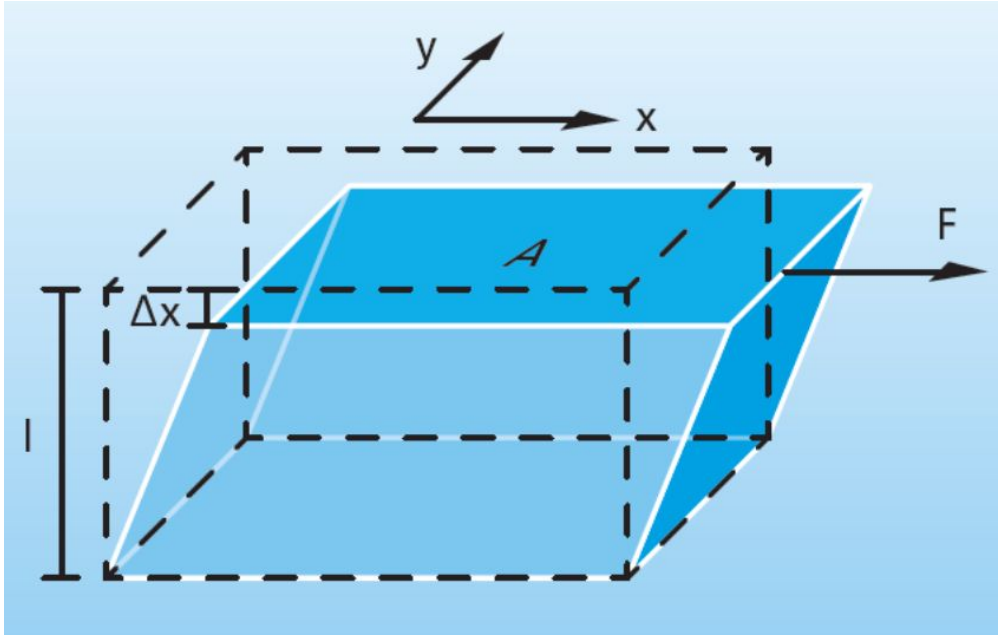
Uma biblioteca de Elementos Finitos feita para pesquisadores de várias áreas do conhecimento.



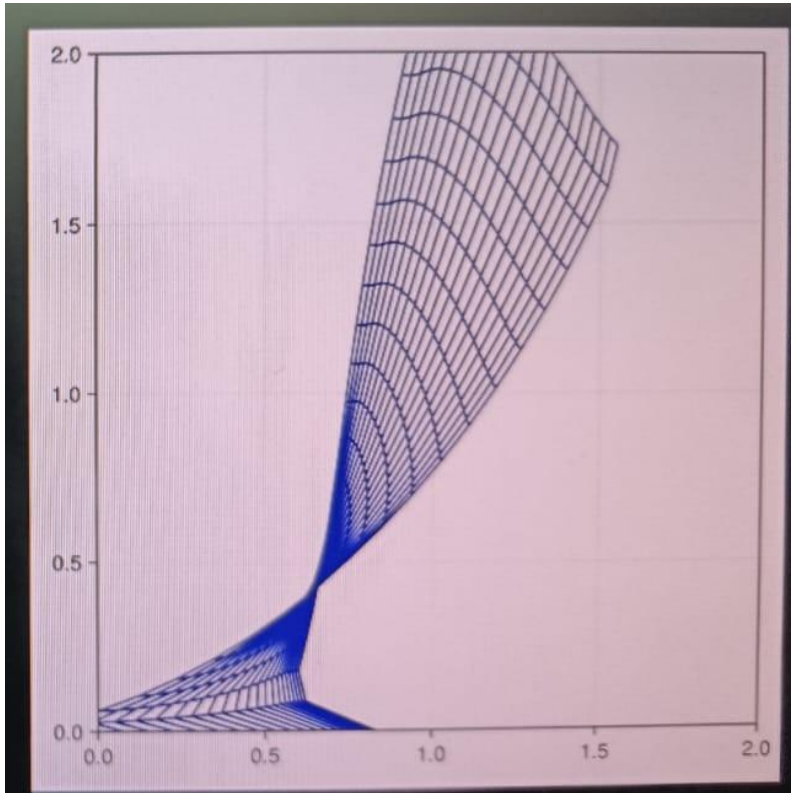
Obrigado!

Perguntas?

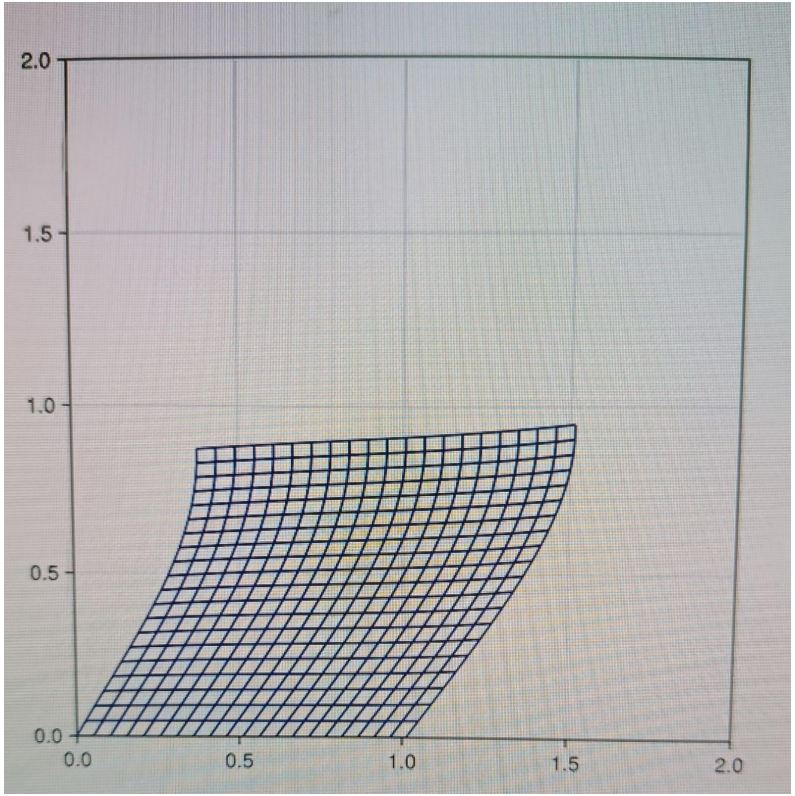
carlosevm@ic.ufrj.br



Extra (experimentos intermediários)

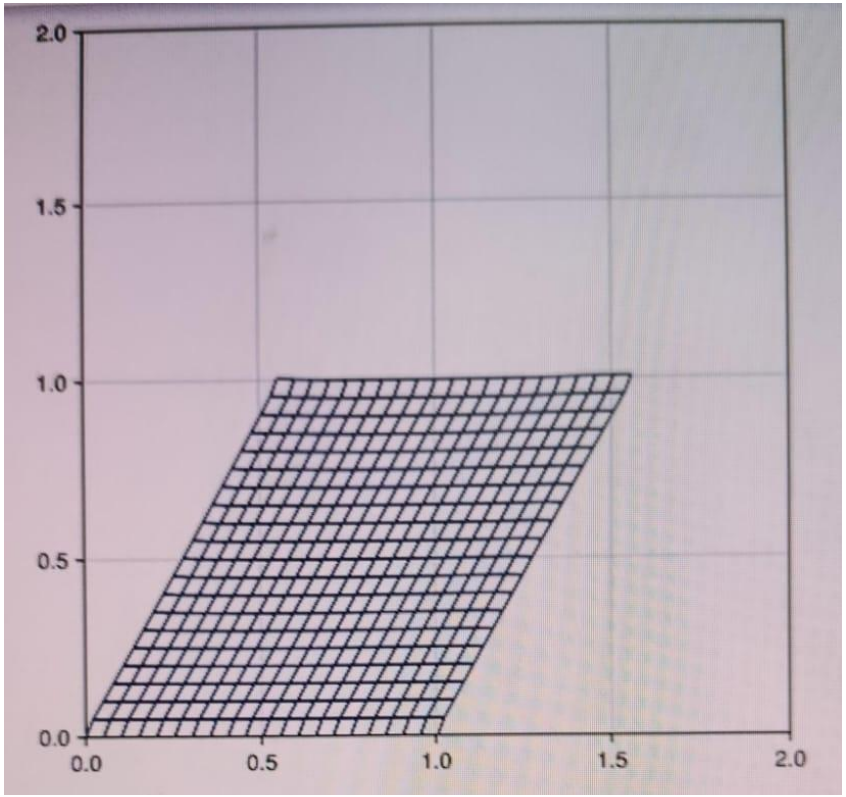


Sinal errado do parâmetro β

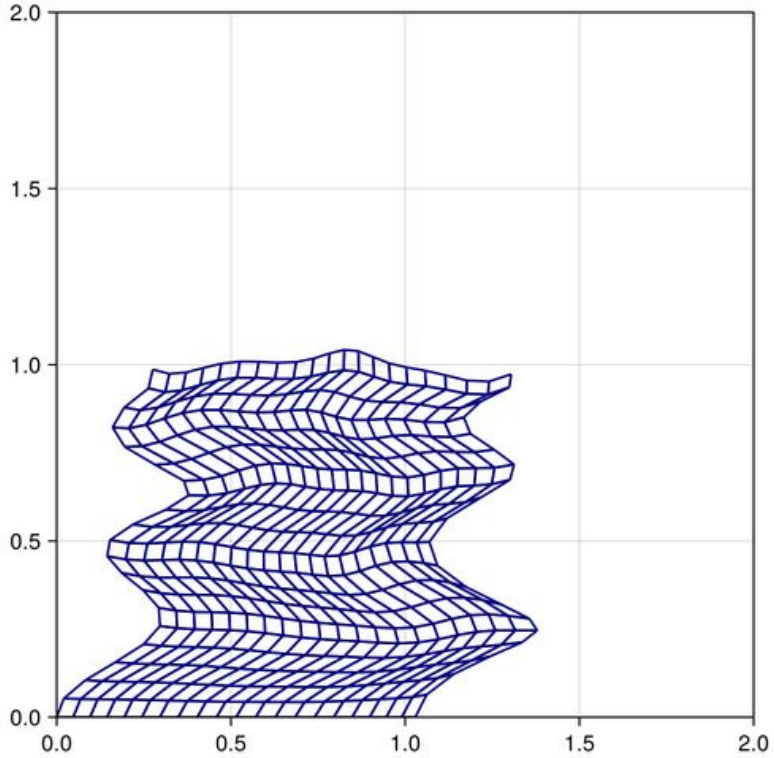


Condição de contorno com direções a mais

Extra (experimentos intermediários)

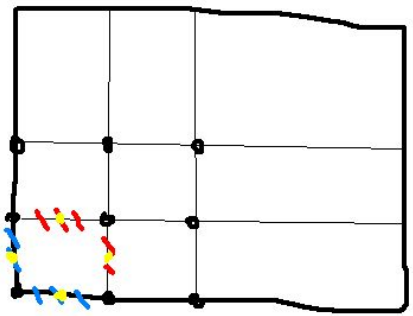


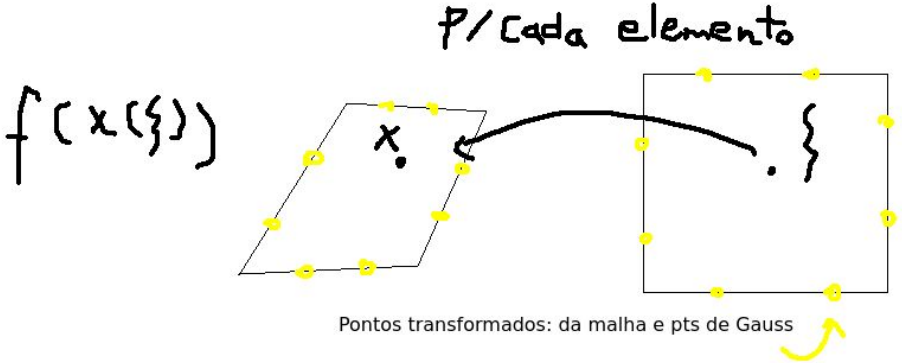
K afetada por erro no cálculo da atualização ALI



Problema na avaliação de f no contorno

Sobre a condição de Neumann





$$\int_{\text{lado1}} f\,\ldots \neq 0 \;+\; \int_{\text{lado2}} f\,\ldots = 0 \;+\; \int_{\text{lado3}} f\,\ldots = 0 \;+\; \int_{\text{lado4}} f\,\ldots \neq 0 \;=\; \int_{\Gamma_1} f\,\ldots$$

No código:

```
if(length(boundaries) != 0)
    for b in 1:4
        ponto = (Xᵉ[b, 1], Xᵉ[b, 2])
        #display(f(ponto))

        for (ξ, w) in zip(P, W)
            termo1 = 0.0

            # Integral lado 1
            p1 = Xᵉ[1, :]
            p2 = Xᵉ[2, :]

            x1_ξ1 = ξ_to_x1(ξ, -1.0)
            x2_ξ2 = ξ_to_x2(ξ, -1.0)

            tamanho_lado = sqrt((p2[1] - p1[1])^2 + (p2[2] - p1[2])^2)
            termo1 += w * φ(a)(ξ, -1.0) * φ(b)(ξ, -1.0) * f([x1_ξ1, x2_ξ2])[r] * tamanho_lado/2
                                                        # fator 2 = tamanho do lado
                                                        # do elemento padrão
```